

Code Injection and Dynamic Evaluation Vulnerabilities in a Node.js Web Application

Bhavana Savaram

University of the Cumberland

MSCS-535-A01: Secure Software Development

Dr. Brandon Bass

September 19, 2026

Code Injection and Dynamic Evaluation Vulnerabilities in a Node.js Web Application

This report documents a set of common JavaScript security vulnerabilities in web applications, demonstrates how each one can be exploited, and describes the mitigation implemented to resolve it. Two vulnerability classes are examined: cross-site scripting (XSS), in which untrusted input is interpreted as executable markup by a web browser, and insecure dynamic code evaluation, in which untrusted input is interpreted as executable JavaScript by a runtime environment. Both vulnerability classes fall under the injection category of the OWASP Top 10 (OWASP Foundation, 2021) and are catalogued individually by the Common Weakness Enumeration (MITRE Corporation, n.d.-a, n.d.-b).

To make each vulnerability and its corresponding fix directly comparable, a small Node.js application was built using the Express framework (OpenJS Foundation, n.d.-a). The application exposes a vulnerable implementation and a secure implementation of each feature discussed, so that identical input can be submitted to both and the difference in behavior observed directly. Findings throughout this report were verified by executing the application and capturing the resulting output rather than by inspecting the source code alone; the automated test results and screenshots included below were produced by that execution. The complete source code for the application is available in a public GitHub repository (Savaram, 2026).

Each of the following sections pairs an explanation of the vulnerability with the vulnerable code, the exact input used to trigger it, the resulting output, and a screenshot captured from the running application (`/vulnerable/*` routes), alongside the same comparison for the corresponding fix (`/secure/*` routes).

Cross-Site Scripting Vulnerabilities

An attacker's script runs inside a victim's browser whenever untrusted input is written into an HTML response or into the Document Object Model (DOM) without the browser being informed that the data should be treated as text rather than markup (OWASP Foundation, n.d.-b). Three variants of this vulnerability were implemented for this report: a server-side reflected variant, a client-side DOM-based variant, and a variant that does not rely on a `<script>` tag at all, since a defense that only targets one payload shape is commonly incomplete.

Reflected Cross-Site Scripting

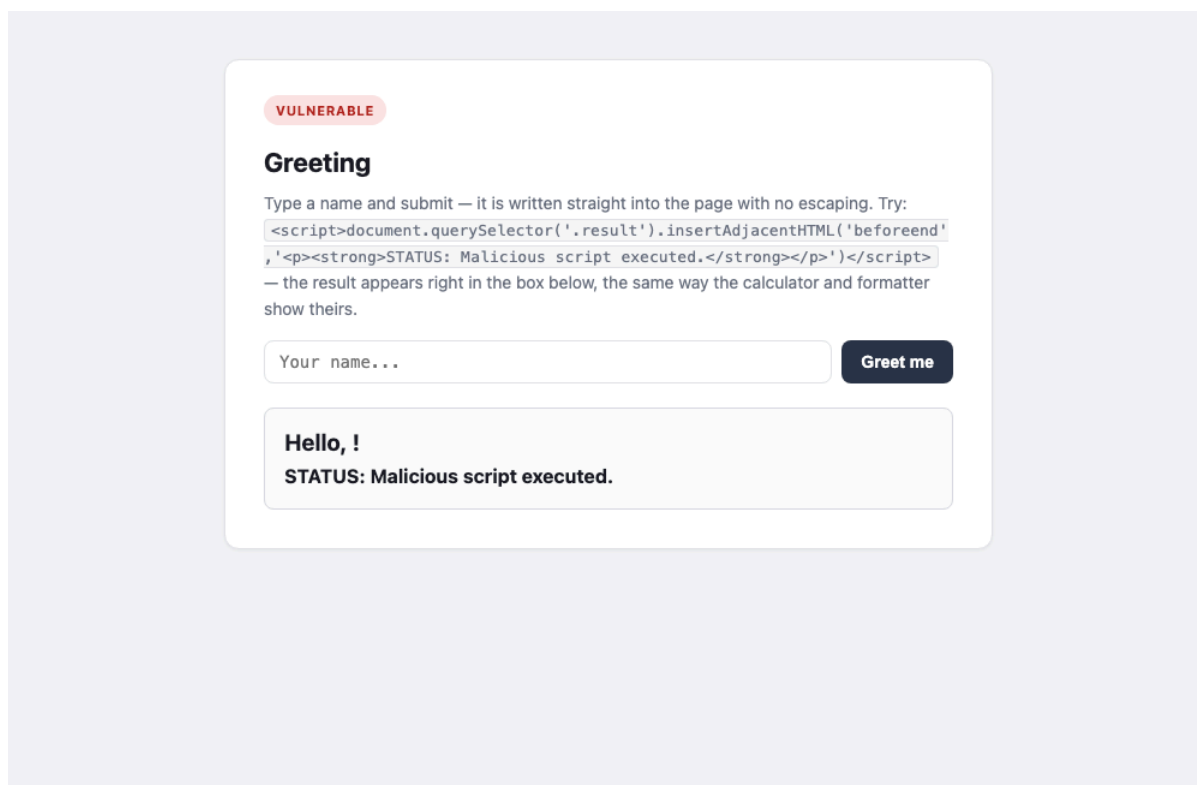
The vulnerable code concatenates a query parameter directly into the HTML response:

```
const name = req.query.name || "friend";
res.send(`... <h1>Hello, ${name}!</h1> ...`);
```

The following request was used to trigger the vulnerability:

```
GET
/vulnerable/greet?name=<script>document.title='XSS-EXECUTED';document.que
rySelector('.result').insertAdjacentHTML('beforeend','<p><strong>STATUS:
Malicious script executed.</strong></p>');</script>
```

The server responded with HTTP 200 and returned the `<script>` tag byte-for-byte unescaped, so a real browser parses and runs it. Figure 1 shows the browser after loading that URL. The status line and the changed tab title were both produced by the attacker's own script rather than by the application. In a real deployment, the same payload could instead read `document.cookie` or call authenticated APIs as the victim.

Figure 1*Vulnerable Reflected XSS Executing in the Browser*

DOM-Based Cross-Site Scripting

In this variant the server plays no part; the bug lies entirely in client-side JavaScript that writes untrusted data into the DOM:

```
const query = new URLSearchParams(window.location.search).get("q") || "";
results.innerHTML = "You searched for: " + query; // untrusted sink
```

The input used was:

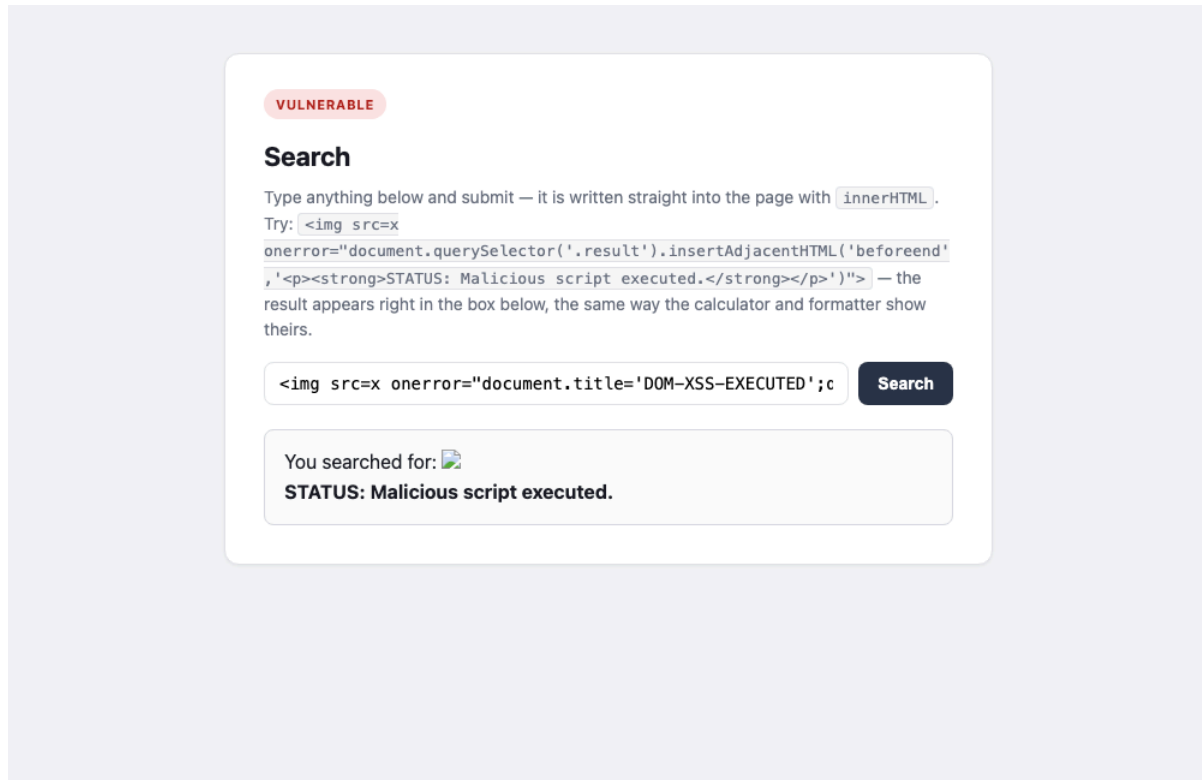
```
/vulnerable/dom-xss.html?q=<img src=x
onerror="document.title='DOM-XSS-EXECUTED';document.querySelector('.result').insertAdjacentHTML('beforeend', '<p><strong>STATUS: Malicious script executed.</strong></p>');">
```

The browser parses the string as HTML, creates a real `<img>` element, fails to load `src=x`, and fires `onerror`, which runs the embedded JavaScript (Figure 2). This variant matters

separately from the reflected variant because it shows that a page can be exploitable even with perfect server-side output encoding if client-side code ever reintroduces an unsafe sink such as `innerHTML`.

Figure 2

Vulnerable DOM-Based XSS Executing in the Browser



Attribute and Event-Handler Injection

The same reflected route is also exploitable with no `<script>` tag anywhere in the payload. The input used was:

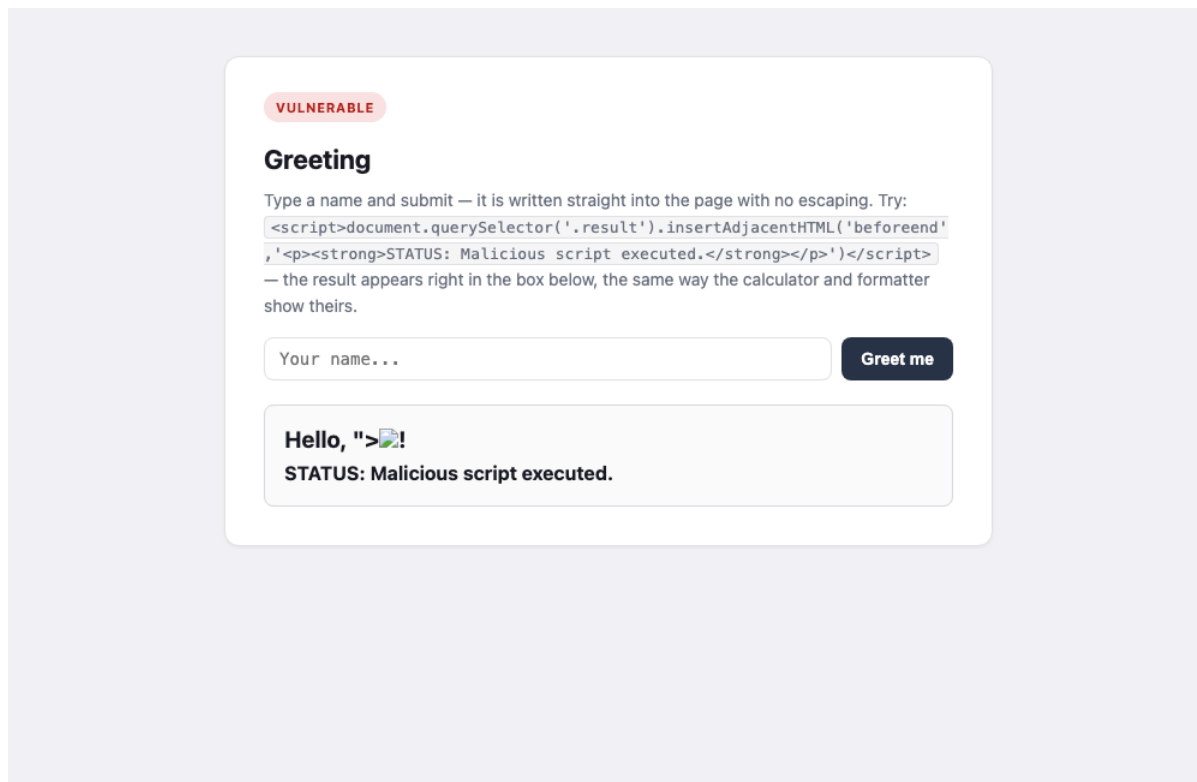
```
GET /vulnerable/greet?name="><img src=x
onerror="document.title='ATTR-XSS-EXECUTED';document.querySelector('.result').insertAdjacentHTML('beforeend', '<p><strong>STATUS: Malicious script
executed.</strong></p>');">
```

The server responded with HTTP 200 and reflected the `<img>` tag unescaped, using the same mechanism as in the reflected variant (Figure 3). This matters because a fix that only

blocks the literal word “script” would miss this payload entirely. The mitigation described later in this report has to encode every dangerous character rather than pattern-match known payload shapes.

Figure 3

Vulnerable Attribute-Breakout Injection Executing Without a Script Tag



Dynamic Code Evaluation Vulnerabilities

JavaScript is an interpreted language, and both Node.js and web browsers expose direct access to their own interpreters at runtime through functions such as `eval()` and the `Function` constructor (MITRE Corporation, n.d.-b). Supplying unvalidated user input to these functions means that the feature being implemented and the execution of arbitrary code become the same code path. Three such features were implemented and exploited for this report: two that invoke

the interpreter explicitly (`eval()` and `Function`) and one that invokes it implicitly (`setTimeout` with a string argument).

Interpreted Language Features That Enable Dynamic Evaluation

An interpreted language is one in which a lower layer of machine code reads and interprets the commands of a higher-level language while the program runs (Richardson & Thies, 2012). Web applications add a further step because their code can be compiled just in time (JIT), which allows dynamic instructions from other modules to be compiled immediately before each execution. Richardson and Thies (2012) identified that trait as the reason JIT compilation is particularly vulnerable to code injection. Both conditions hold in the two runtimes used in this report: Node.js runs the server-side code and the browser runs the client-side code, and each keeps a compiler and interpreter available while the application executes.

Three features of this design make dynamic evaluation possible, and each is demonstrated in the sections that follow. First, the runtime exposes its compiler to the program as ordinary functions, so source text held in a string can be turned into running code on demand. Second, code and data share one representation: to the runtime, a string that arrived in a form field is indistinguishable from a string that a developer wrote. Third, the scope in which the evaluated code runs depends on the feature used. The `eval()` function runs its argument in the caller's lexical scope, the `Function` constructor compiles a new function that sees only the global scope, and the browser's timer functions compile a string argument in the global scope when the timer fires. Table 1 summarizes these differences alongside the fix applied to each feature later in this report.

Table 1*Dynamic Evaluation Features Demonstrated in This Report*

Feature	Invocation	Scope of evaluated code	Where it runs	Fix used in this report
<code>eval()</code>	Explicit	Caller's lexical scope	Node.js and browser	Allowlist arithmetic parser (<code>safeMathEval.js</code>)
<code>new Function()</code>	Explicit	Global scope	Node.js and browser	Fixed map of named formatters (<code>formatters.js</code>)
<code>setTimeout(string)</code>	Implicit	Global scope	Browser only	Function reference instead of a string (<code>dynamic-timer-safe.js</code>)

These features are not unique to JavaScript. The Common Weakness Enumeration entry for eval injection lists JavaScript, Python, Perl, PHP, and Ruby among the languages in which the weakness occurs (MITRE Corporation, n.d.-b), and Richardson and Thies (2012) singled out the `eval` function in Perl as the point at which malicious input becomes the next line of code in the program. The underlying rule is therefore independent of language, and it is the first of three rules the textbook gives for JIT-compiled systems: never execute your input (Richardson & Thies, 2012). The remaining two rules, keeping a layer between code and input and mapping every exception, guide the fixes described in the mitigation sections.

Dynamic Evaluation With `eval()`

```
const { expression } = req.body;
const result = eval(expression);
```

The code was intended as a calculator (`2+2` returns `4`), but `eval()` runs full JavaScript rather than arithmetic. It can also read variables outside the function's own inputs because it executes in the enclosing lexical scope. Table 2 lists four requests made against the running server, and Figure 4 shows the same requests captured live by a small `fetch()`-driven test console, with the vulnerable and secure routes side by side.

Table 2*Results of Four eval() Requests Against the Vulnerable Route*

No	Scenario	Input	Vulnerable output
1	Legitimate arithmetic	{"expression":"2 + 3 * 4"}	200 {"result":14}
2	Read an out-of-scope secret	{"expression":"INTERNAL_API_KEY"}	200 {"result":"sk-demo-INTERNAL-SECRET-DO-NOT-LEAK"}
3	Arbitrary code execution (IIFE)	{"expression":"(function(){return 1+1===2 ? 'RCE-PROOF' : 'no';})();"}	200 {"result":"RCE-PROOF"}
4	Reach Node internals	{"expression":"process.version"}	200 {"result":"v25.9.0"}

Figure 4*Calculator Scenarios Using eval() Run Against the Vulnerable and Secure Routes*

Scenario	Request body	POST /vulnerable/calculate	POST /secure/calculate
Legitimate arithmetic	{"expression":"2 + 3 * 4"}	HTTP 200 { "result": 14 }	HTTP 200 { "result": 14 }
Read out-of-scope secret	{"expression":"INTERNAL_API_KEY"}	HTTP 200 { "result": "sk-demo-INTERNAL-SECRET-DO-NOT-LEAK" }	HTTP 400 { "error": "Unexpected character at position 0: \"\ \" " }
Arbitrary code execution (IIFE)	{"expression":"(function(){return 1+1===2 ? 'RCE-PROOF' : 'no';})();"}	HTTP 200 { "result": "RCE-PROOF" }	HTTP 400 { "error": "Unexpected character at position 1: \"f\" " }
Reach Node internals	{"expression":"process.version"}	HTTP 200 { "result": "v25.9.0" }	HTTP 400 { "error": "Unexpected character at position 0: \"p\" " }

Row 2 of Table 2 is the key proof: `INTERNAL_API_KEY` is a module-level constant unrelated to the calculator, and it exists purely to show that `eval()` can read data the API was never designed to expose. Row 4 shows that the same primitive reaches the Node.js process itself.

Beyond the API results above, this scenario also has a real interactive page (`/vulnerable/calculator.html` and `/secure/calculator.html`) rather than only a JSON endpoint. Figures 5 and 6 show `INTERNAL_API_KEY` typed directly into the on-page form and submitted to the vulnerable and secure versions, respectively.

Figure 5

Vulnerable Calculator Page Leaking `INTERNAL_API_KEY` Through the On-Page Form

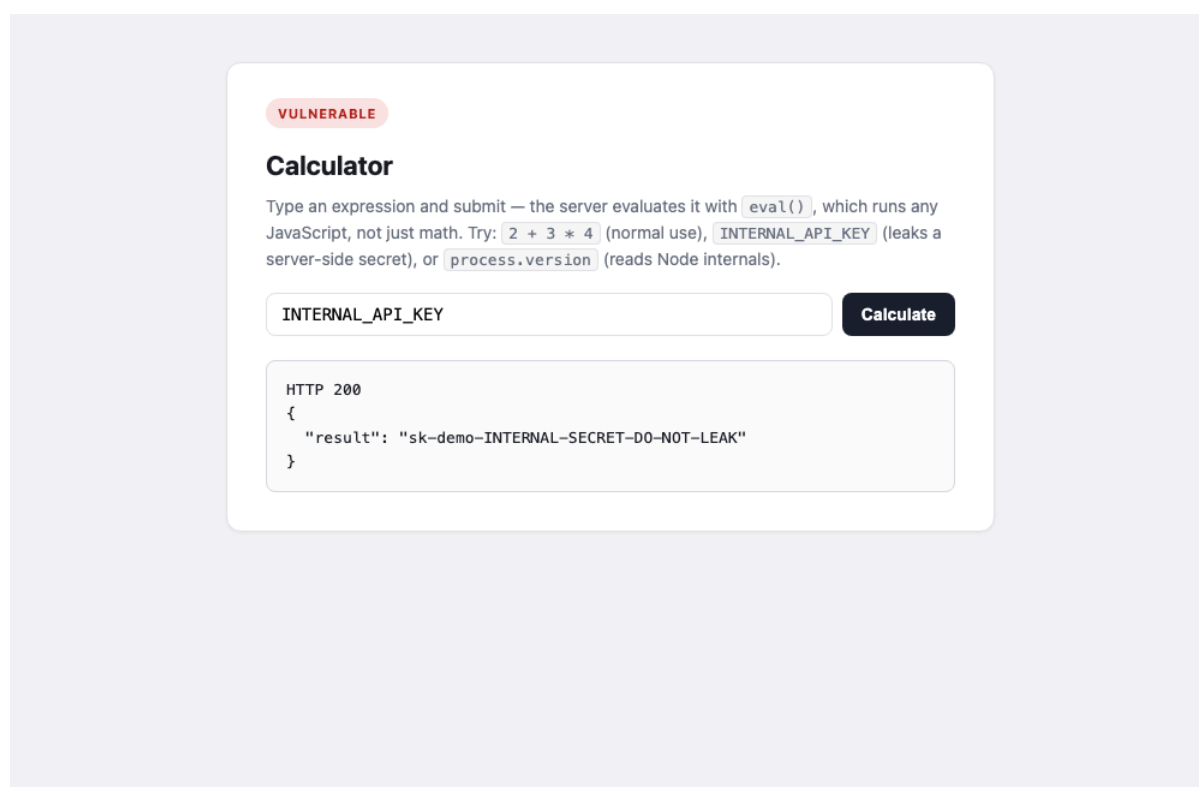


Figure 6

Secure Calculator Page Rejecting the Same Input With a 400 Error



Dynamic Evaluation With the Function Constructor

```
const fn = new Function("n", formatter); // formatter is
attacker-controlled
fn(value);
```

This code implements a “number formatter” feature that compiles the client’s expression into a real function. Unlike `eval()`, a `Function` body runs against the global scope, which is why this variant is used to show the primitive reaching `process` directly rather than through scope leakage. Table 3 lists two requests made against the running server, and Figure 7 shows the same requests run against both routes.

Table 3

Results of Two Function Constructor Requests Against the Vulnerable Route

No.	Scenario	Vulnerable input	Vulnerable output
1	Legitimate formatter	<code>{"formatter":"return n.toFixed(2);","value":3.14159}</code>	200 <code>{"result":"3.14"}</code>
2	Reach Node globals via formatter body	<code>{"formatter":"return typeof process;","value":1}</code>	200 <code>{"result":"object"}</code>

Figure 7

Formatter Scenarios Using the Function Constructor Run Against the Vulnerable and Secure

Routes

Scenario	Request body	POST /vulnerable/format	POST /secure/format
Legitimate formatter (vulnerable route)	vuln: <code>{"formatter":"return n.toFixed(2);","value":3.14159}</code> secure: <code>{"formatterKey":"fixed2","value":3.14159}</code>	HTTP 200 { "result": "3.14" }	HTTP 200 { "result": 3.14 }
Reach Node globals via formatter body	vuln: <code>{"formatter":"return typeof process;","value":1}</code> secure: <code>{"formatterKey":"return typeof process;","value":1}</code>	HTTP 200 { "result": "object" }	HTTP 400 { "error": "Unknown formatterKey. Allowed: fixed2, percent, rounded, currencyUSD" }

The formatter also has a dedicated page (`/vulnerable/formatter.html` and `/secure/formatter.html`). Figures 8 and 9 show `return typeof process;` typed directly into the on-page form and submitted to the vulnerable and secure versions, respectively.

Figure 8

Vulnerable Formatter Page Reaching the Node.js process Object

The screenshot shows a web interface for a 'Vulnerable Formatter'. At the top, a red pill-shaped badge contains the word 'VULNERABLE'. Below this, the title 'Formatter' is displayed. A paragraph explains the vulnerability: 'Enter a number and a formatter body — the server compiles it with `new Function("n", formatter)`, which runs any JavaScript as the function body, not just formatting logic. Try: `return n.toFixed(2);` (normal use) or `return typeof process;` (reaches Node internals).' Below the text are two input fields: 'Number' with the value '1' and 'Formatter body' with the value 'return typeof process;'. A dark 'Format' button is to the right of the second input. Below the inputs, a light gray box displays the HTTP response: 'HTTP 200' followed by a JSON object: `{ "result": "object" }`.

Figure 9

Secure Formatter Page Rejecting the Same Input and Listing the Allowed Formatter Names

The screenshot shows a web application interface for a 'Formatter'. At the top, there is a green 'SECURE' badge. Below it, the title 'Formatter' is displayed. A paragraph explains the feature: 'Same feature as /vulnerable/formatter.html, mitigated. The server only accepts a name from a fixed list of formatters — never code. Try: fixed2, percent, rounded, or currencyUSD (all work), or return typeof process; (rejected)'. Below this text, there are two input fields: 'Number' with the value '1' and 'Formatter key' with the value 'return typeof process;'. To the right of the 'Formatter key' field is a dark blue 'Format' button. Below the inputs, a light gray box displays the JSON response: 'HTTP 400 { "error": "Unknown formatterKey. Allowed: fixed2, percent, rounded, currencyUSD" }'.

Figures 8 and 9 are unedited captures of the real JSON responses from the running application, taken by a page that makes genuine `fetch()` POST requests; nothing on the page is hand-typed.

Implicit Evaluation via `setTimeout`

A third dynamic-evaluation sink is deliberately different from the previous two: no `eval` or `Function` call appears anywhere in the source. According to the HTML timers specification, `setTimeout` and `setInterval` implicitly compile and run a string argument as JavaScript when the timer fires.

```
// public/vulnerable/dynamic-timer.js
const run = new URLSearchParams(window.location.search).get("run") ||
"...";
setTimeout(run, 50); // VULNERABLE: a string callback is compiled and run
```

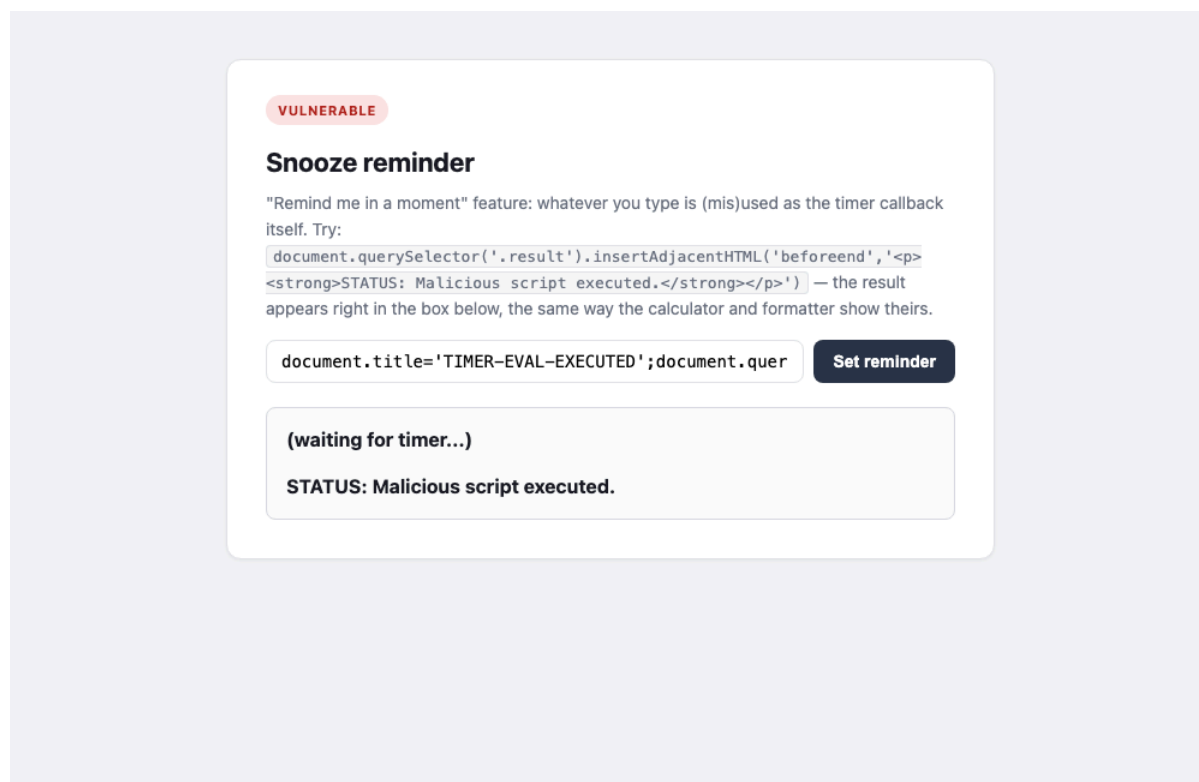
The input used was:

```
/vulnerable/dynamic-timer.html?run=document.title='TIMER-EVAL-EXECUTED';document.querySelector('.result').insertAdjacentHTML('beforeend','<p><strong>STATUS: Malicious script executed.</strong></p>');
```

After 50 ms, the browser compiles and runs that string (Figure 10).

Figure 10

Vulnerable setTimeout Evaluating a String Payload



This sink is browser-only. The `setTimeout` function in Node.js requires a real function and throws an error when given a string, so this sink can appear only in client-side code. The behavior was proven directly with a real DOM (jsdom) in `tests/settimeout-eval.test.js`, since Node.js timers cannot reproduce the string-evaluation behavior documented for the browser API (Mozilla Contributors, n.d.-b).

Security Mitigations and Content Security Policy

Each vulnerability described in the preceding sections has a corresponding fix implemented on a parallel set of routes, exercised here against the identical input used to demonstrate the original vulnerability.

Output Encoding

The `escapeHtml()` function HTML-entity-encodes `name` before it reaches the response, so `<script>` becomes the literal text `<script>` and can never be parsed as markup. The same input was then sent to the secure route:

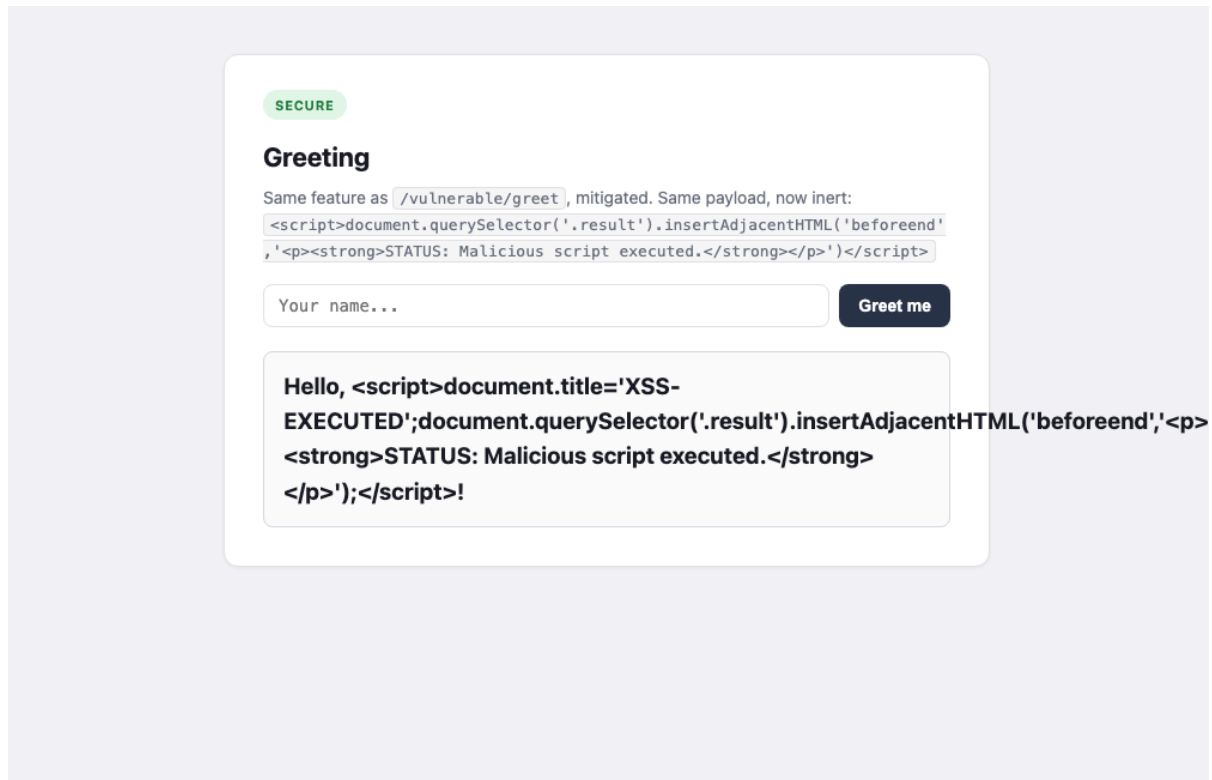
```
GET /secure/greet?name=<script>document.title='XSS-EXECUTED';...</script>
```

As Figure 11 shows, no status line appears and the page title stays “Secure Greeting.”

The exact same input that executed earlier is now inert text on screen.

Figure 11

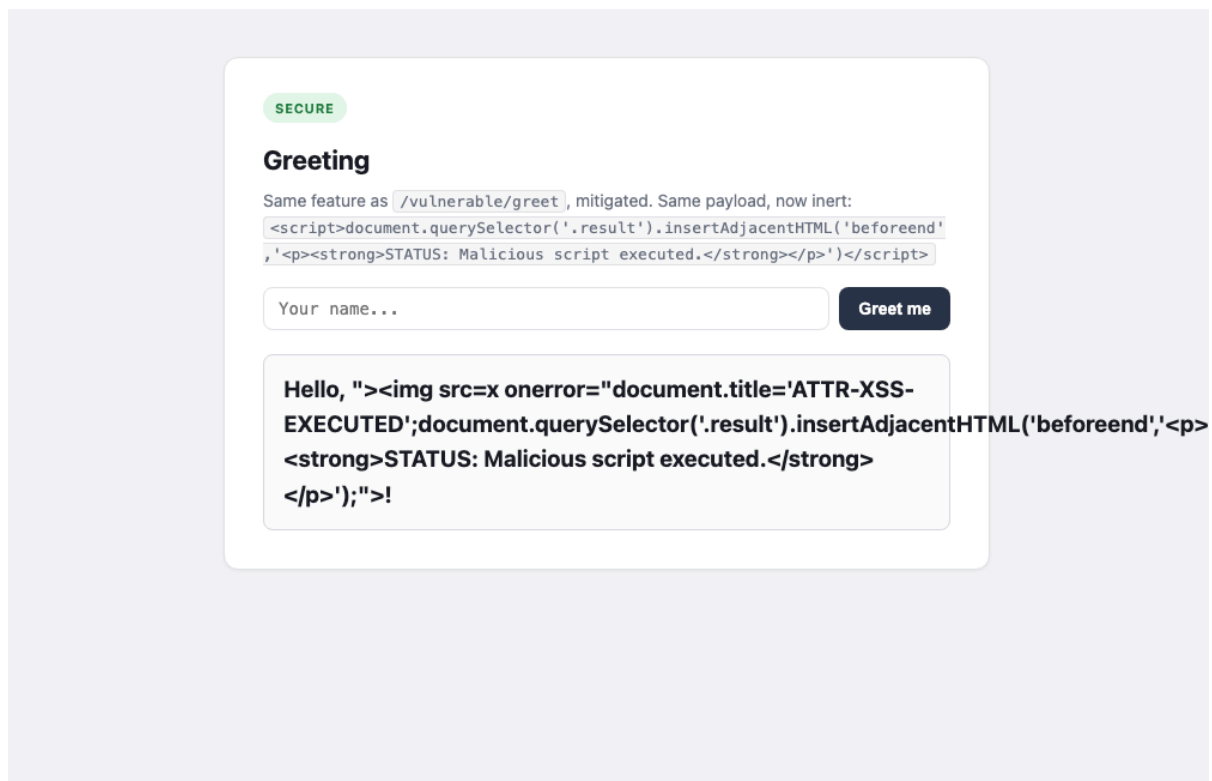
Secure Reflected Route Rendering the Payload as Escaped Text



The same `escapeHtml()` call handles the attribute-breakout payload as well, with no extra code, because it encodes `<` and `>` unconditionally, regardless of which tag name follows them (Figure 12).

Figure 12

Secure Attribute-Breakout Payload Rendered as Escaped Text

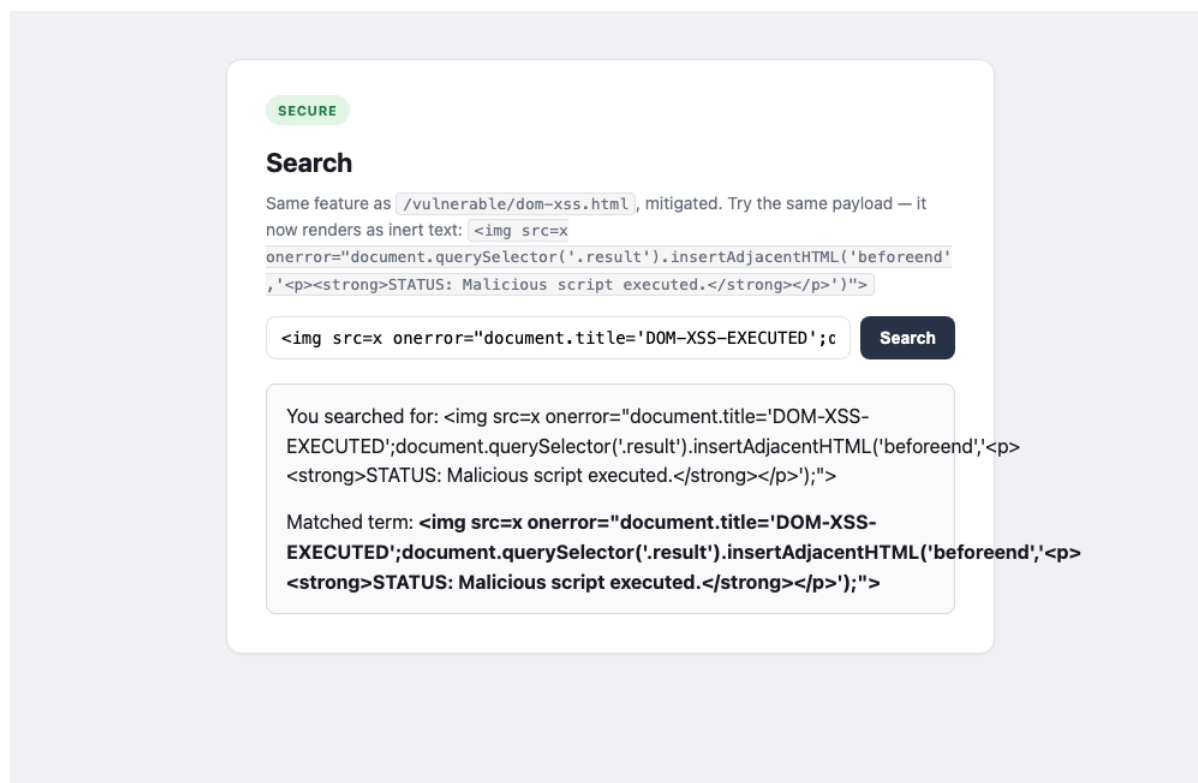


Safe DOM Sinks and Output Sanitization

The `dom-safe.js` module uses `textContent` (never parsed as HTML) for plain text, and `DOMPurify.sanitize()` with an explicit tag allowlist for the one place where real HTML output is wanted (bolding a matched term). Figure 13 shows the same `img/onerror` payload rendered as literal text, with no status line appearing.

Figure 13

Secure DOM-Based Route Rendering the Payload as Literal Text



Replacing eval() With an Allowlist Parser

The `safeMathEval.js` module is a hand-written recursive-descent parser whose tokenizer recognizes only numbers and the operators `+` `-` `*` `/` `(` `)`. Because there is no token type for an identifier, dot, or bracket, `INTERNAL_API_KEY`, `process.version`, and `(function(){...})()` all fail to tokenize and are rejected with a 400 status before evaluation starts (see the `/secure/calculate` column in Figure 4 and the rejected form submission in Figure 6). Legitimate arithmetic still returns the identical result as the vulnerable route. The tokenizer acts as the layer that the textbook recommends placing between code and input: only tokens it recognizes ever reach evaluation (Richardson & Thies, 2012). Returning a 400 response for every unrecognized token also maps the exception case explicitly, so input outside the

expected bounds has a defined outcome rather than being passed along (Richardson & Thies, 2012).

Replacing the Function Constructor With a Named Allowlist

The `formatters.js` module replaces “send me a JS expression” with a fixed map of four named, developer-written functions (`fixed2`, `percent`, `rounded`, and `currencyUSD`). The client sends a name, never code, and the server never compiles anything at request time (see the `/secure/format` column in Figure 7 and the rejected form submission in Figure 9). Because the only value that crosses from the client into the evaluation path is a name looked up in a fixed map, an attacker has no path for supplying code, and an unrecognized name is rejected with an error that lists the allowed names.

Always Passing `setTimeout` a Function Reference

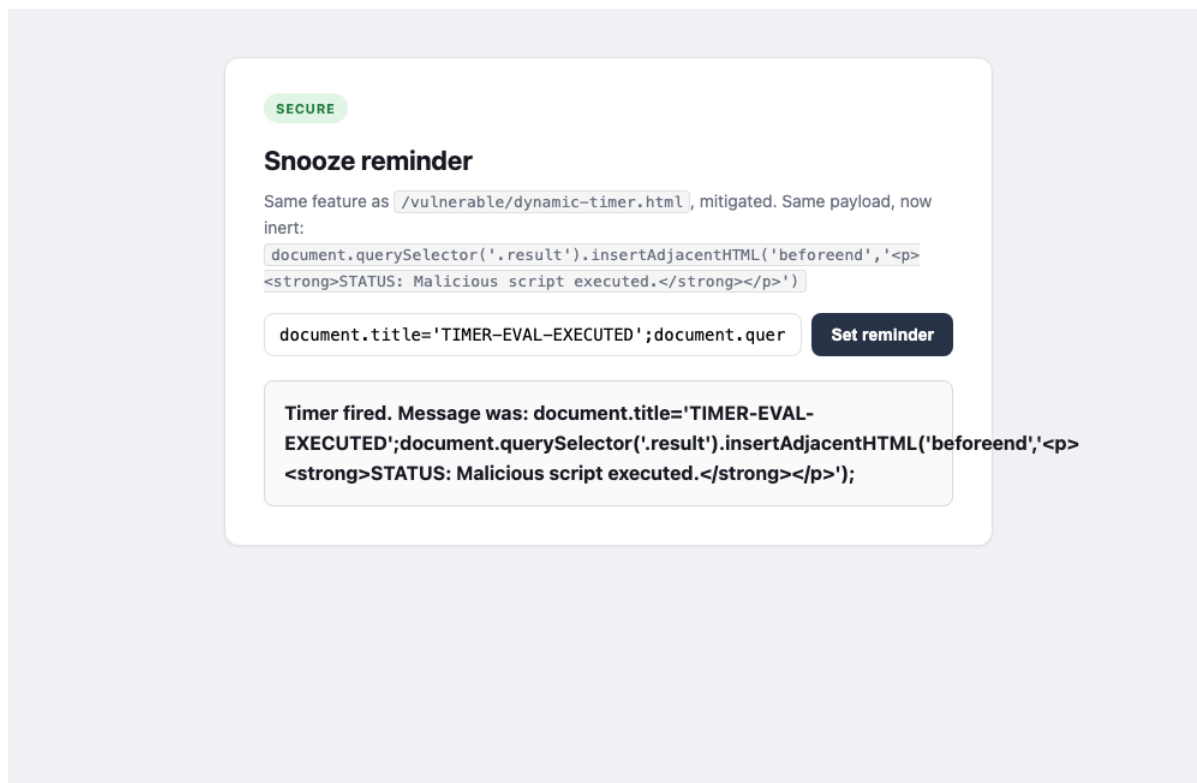
The `dynamic-timer-safe.js` file never passes a string to `setTimeout`; the user’s input is only ever read by the callback (displayed as text) and is never compiled as the callback itself:

```
setTimeout(function () {
  document.getElementById("status").textContent = "Timer fired. Message
was: " + run;
}, 50);
```

No validation step is needed here. Once the callback is a real function literal, there is no longer any string for the browser to compile, so no payload can reach the interpreter through this path. Figure 14 shows the same payload rendered as visible text, with no status line and an unchanged title.

Figure 14

Secure setTimeout Page Rendering the Payload as Visible Text



Content Security Policy as Defense in Depth

Even if an escaping bug were reintroduced later, a strict Content Security Policy (CSP) limits what an injected payload can do. A CSP is an HTTP response header that tells the browser which sources of scripts, styles, images, and other content it may load and run (Mozilla Contributors, n.d.-a). The policy used on the secure routes sets `script-src 'self'` 'nonce-`<per-request>`', with no `unsafe-inline` and no `unsafe-eval`. In a nonce-based policy, the server generates a fresh random value for each response, places it in the header, and adds the same value to the `nonce` attribute of the scripts it intends to allow. The browser then runs only scripts that come from the same origin or carry the matching nonce, so a script injected

by an attacker is refused because the attacker cannot predict a value that changes with every response.

Omitting `unsafe-inline` and `unsafe-eval` matters for this project in particular. Without `unsafe-inline`, a browser that enforces the policy refuses inline scripts and inline event-handler attributes, which are the delivery mechanisms for the `<script>` and `onerror` payloads shown earlier. Without `unsafe-eval`, the browser also refuses the string-to-code mechanisms examined in the previous section: `eval()`, the `Function` constructor, and `setTimeout` with a string argument (Mozilla Contributors, n.d.-a). The policy does not replace the fixes described above, but it backs up both groups of them if one is later bypassed (OWASP Foundation, n.d.-a).

In this application the nonce is generated fresh with `crypto.randomBytes(16)` on every request. The directive set follows the guidance for a restrictive, allowlist-based policy described by Mozilla Contributors (n.d.-a) and OWASP Foundation (n.d.-a). Figure 15 compares the response headers of the two greeting routes, and Table 4 lists the header value returned by each.

Figure 15

Content-Security-Policy Headers Returned by the Vulnerable and Secure Greeting Routes

Route	Content-Security-Policy response header
<code>/vulnerable/greet?name=Ada</code>	(not set)
<code>/secure/greet?name=Ada</code>	<code>default-src 'self';script-src 'self' 'nonce-sYj1+CX123l4/R3/tnbibQ==';style-src 'self';img-src 'self' data;;connect-src 'self';object-src 'none';base-uri 'self';frame-ancestors 'none';form-action 'self';upgrade-insecure-requests</code>

Table 4*Content-Security-Policy Header by Route*

Route	Content-Security-Policy header
GET /vulnerable/greet?name=Ada	(not set)
GET /secure/greet?name=Ada	default-src 'self'; script-src 'self' 'nonce-...'; style-src 'self'; img-src 'self' data:; connect-src 'self'; object-src 'none'; base-uri 'self'; frame-ancestors 'none'; form-action 'self'; upgrade-insecure-requests

Table 5 explains what each directive in the secure policy does and why it is included.

Three of them, `base-uri`, `form-action`, and `frame-ancestors`, do not fall back to `default-src`, which is why the policy lists each one explicitly (Mozilla Contributors, n.d.-a).

Table 5*Directives in the Secure Content Security Policy and What Each Does*

Directive	Value	What it does and why it is used
<code>default-src</code>	'self'	Fallback for every resource type that has no directive of its own; anything not listed below may load only from the same origin.
<code>script-src</code>	'self' 'nonce-...'	Allows scripts only from the same origin or with the per-request nonce; inline scripts without the nonce, inline event handlers, and string-to-code functions such as <code>eval()</code> are refused.
<code>style-src</code>	'self'	Allows stylesheets only from the same origin; injected <code><style></code> blocks and inline <code>style</code> attributes are refused.
<code>img-src</code>	'self' data:	Allows images from the same origin and from inline data: URIs.
<code>connect-src</code>	'self'	Limits the targets of <code>fetch()</code> , <code>XMLHttpRequest</code> , and <code>WebSocket</code> connections to the same origin, which restricts one common route for sending stolen data to an attacker.
<code>object-src</code>	'none'	Blocks plugin content such as <code><object></code> and <code><embed></code> , closing plugin-based injection.
<code>base-uri</code>	'self'	Restricts the URLs a <code><base></code> element may set, which stops a <code><base></code> -tag hijack of relative URLs.
<code>frame-ancestors</code>	'none'	Prevents other sites from embedding the page in a frame, which blocks clickjacking.
<code>form-action</code>	'self'	Restricts where forms on the page may submit, so an injected form cannot post data to another origin.

Directive	Value	What it does and why it is used
<code>upgrade-insecure-requests</code>	<i>(no value)</i>	Instructs the browser to rewrite HTTP URLs to HTTPS before requesting them.

Testing and Verification

Running `npm test` executes 41 tests across six suites using Jest (OpenJS Foundation, n.d.-b) and Supertest, and all of them pass. Table 6 summarizes what each suite demonstrates.

Table 6

Jest Test Suites and What Each Suite Proves

Suite	Proves
<code>xss.test.js</code>	Reflected-XSS payload returned unescaped by <code>/vulnerable/greet</code> , escaped by <code>/secure/greet</code>
<code>dom-injection.test.js</code>	<code>innerHTML</code> turns the payload into a real, attribute-bearing DOM element; <code>textContent</code> does not; <code>DOMPurify.sanitize()</code> strips the dangerous parts while keeping allowlisted tags
<code>eval.test.js</code>	<code>eval()</code> and <code>new Function()</code> execute arbitrary JavaScript, including leaking the out-of-scope secret; the fixed routes reject the same payloads
<code>safeMathEval.test.js</code>	The eval-free arithmetic parser is correct on legitimate expressions and rejects every non-arithmetic input
<code>csp.test.js</code>	The CSP header's directives are present, <code>unsafe-inline</code> and <code>unsafe-eval</code> are absent, and the nonce changes on every request
<code>settimeout-eval.test.js</code>	<code>setTimeout(string, delay)</code> is compiled and executed as code (a third, browser-only dynamic-evaluation sink, distinct from <code>eval</code> and <code>Function</code>); always passing a real function fixes it

Every test in a “VULNERABLE” describe block is expected to demonstrate the exploit succeeding. That result is the executable proof that the vulnerability is real, rather than merely a description of one.

Project Structure and Execution Instructions

The runnable application is published in the `project-2` folder of a public GitHub repository (Savaram, 2026). Table 7 lists the files and folders it contains.

Table 7*Contents of the project-2 Folder in the GitHub Repository*

Path	Contents
project-2/src/routes/vulnerable.js	Reflected XSS and <code>eval()/new Function()</code> endpoints
project-2/src/routes/secure.js	Fixed counterparts
project-2/src/middleware/csp.js	Strict, nonced Content Security Policy
project-2/src/utils/	<code>escapeHtml.js</code> , <code>safeMathEval.js</code> , <code>formatters.js</code>
project-2/public/home/index.html	Homepage that links to every scenario
project-2/public/shared/styles.css	Shared page styling (badges, cards, forms)
project-2/public/vulnerable/ and project-2/public/secure/	DOM XSS and <code>setTimeout(string)</code> demonstration pages, each with a form for trying custom input
project-2/public/scenarios/console.html	Live <code>fetch()</code> -driven scenario dashboard
project-2/tests/	The 41-test Jest and Supertest suite
project-2/scripts/capture-screenshots.js	Script that regenerates every screenshot in this report

To run the application, clone the repository, install its dependencies, start the server, and run the tests with the following commands:

```
git clone https://github.com/BhavanaSavaram/secure-software-development.git
cd secure-software-development/project-2
npm install    # also vendors DOMPurify via postinstall
npm start     # http://localhost:3000
npm test      # 41 tests
```

Every screenshot in this report can be regenerated with the following commands:

```
cd secure-software-development/project-2
npx playwright install chromium # first run only
PORT=3100 node src/server.js &
node scripts/capture-screenshots.js
```

Each screenshot is captured with a real, automated Chromium browser (Playwright) after navigating to the exact URL shown. Because a headless browser has no operating system window frame to capture, the capture script adds one visual element that is not part of the application itself: a bar at the top showing the real URL that was requested, styled to read like a

browser's address bar. Everything below that bar (the page, the form, and the result) is the application's own unmodified output.

References

- MITRE Corporation. (n.d.-a). *CWE-79: Improper neutralization of input during web page generation* ('Cross-site Scripting'). Common Weakness Enumeration.
<https://cwe.mitre.org/data/definitions/79.html>
- MITRE Corporation. (n.d.-b). *CWE-95: Improper neutralization of directives in dynamically evaluated code* ('Eval Injection'). Common Weakness Enumeration.
<https://cwe.mitre.org/data/definitions/95.html>
- Mozilla Contributors. (n.d.-a). *Content-Security-Policy header*. MDN Web Docs.
<https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Content-Security-Policy>
- Mozilla Contributors. (n.d.-b). *setTimeout() global function*. MDN Web Docs.
<https://developer.mozilla.org/en-US/docs/Web/API/setTimeout>
- OpenJS Foundation. (n.d.-a). *Express* [Computer software]. <https://expressjs.com/>
- OpenJS Foundation. (n.d.-b). *Jest* [Computer software]. <https://jestjs.io/>
- OWASP Foundation. (n.d.-a). *Content security policy cheat sheet*. OWASP Cheat Sheet Series.
https://cheatsheetseries.owasp.org/cheatsheets/Content_Security_Policy_Cheat_Sheet.html
- OWASP Foundation. (n.d.-b). *Cross site scripting prevention cheat sheet*. OWASP Cheat Sheet Series.
https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
- OWASP Foundation. (2021). *A03:2021 – Injection*. OWASP Top 10:2021.
https://owasp.org/Top10/A03_2021-Injection/
- Richardson, T., & Thies, C. N. (2012). *Secure software design*. Jones & Bartlett Learning.

Savaram, B. (2026). *secure-software-development* [Computer software]. GitHub.

<https://github.com/BhavanaSavaram/secure-software-development/tree/main/project-2>